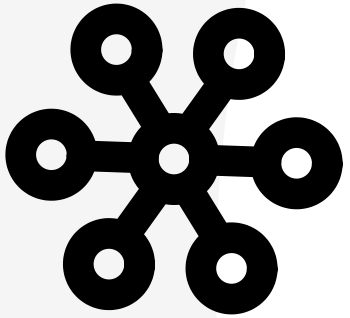
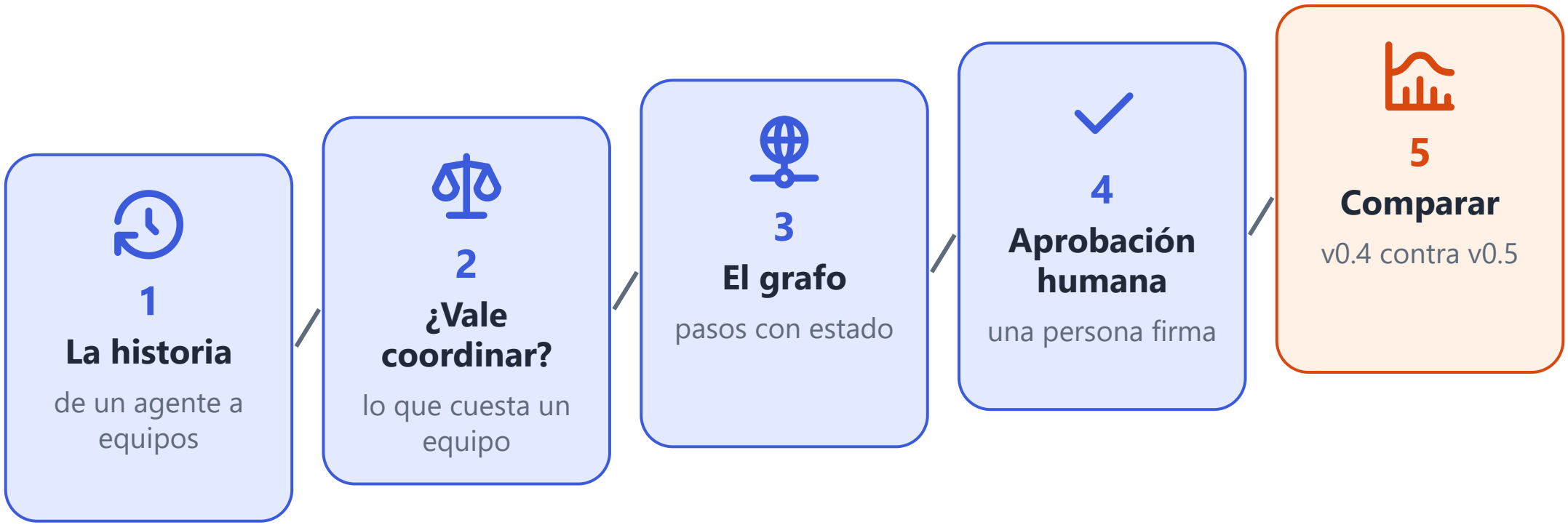
# Varios agentes

## M7: v0.5, Multiagente



Hoy: el recorrido

# De un agente a un equipo que se controla



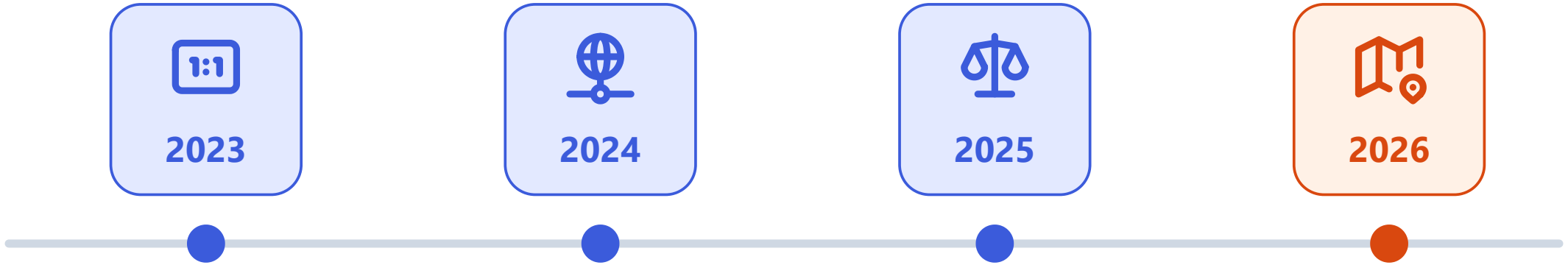

Venís de v0.4. Hoy repartís el trabajo y sumás una persona al final.

# Primero solo, después lo discutimos

- Cuestionario individual (iRAT), 5 min
- Discusión de las respuestas, 5 min

Sin asistentes de IA. 5 preguntas sobre grafos y coordinación. En la discusión, las mismas.

# De un agente a equipos de agentes




**La pregunta pasó de «¿se puede?» a «¿conviene?».**

# LangGraph no es la única opción

**LangGraph**

**Google ADK**

**CrewAI**

**AutoGen / AG2 (Microsoft)**

**OpenAI Agents SDK**

**n8n, sin código**

Mismas ideas: estado, roles, pausa para una persona. No las vemos todas, pero existen.

# Más agentes dieron peor resultado

**Un agente**
**12/12**
**2,7 llamadas**

≠

**Equipo**
**falla la 12 siempre**
**4,8 a 9,2 llamadas**

- Gemini 3.5 Flash-Lite, 3 corridas.
- Modelo chico: 9/12 solo, 3/12 en equipo.
- ¿Por qué?

# Antes de sumar agentes, medí el costo

### Anthropic

un agente, unas 4 veces los tokens de un chat; un multiagente, unas 15

### LangChain

empezá con un solo agente y buenos prompts

### Nuestra suite

2,6 veces las llamadas y peor elección

# Hay cuatro formas de repartir el trabajo

### Subagentes

un supervisor llama a otros agentes

### Skills

carga saberes al necesitarlos

### Handoffs

pasa la posta a otro agente

### Router

clasifica la pregunta y la reparte

`v0.5` usa el primero, como grafo propio (*custom workflow*).

# Un grafo: un diagrama de flujo que corre



### Nodo

cada caja: una función de Python



### Flecha

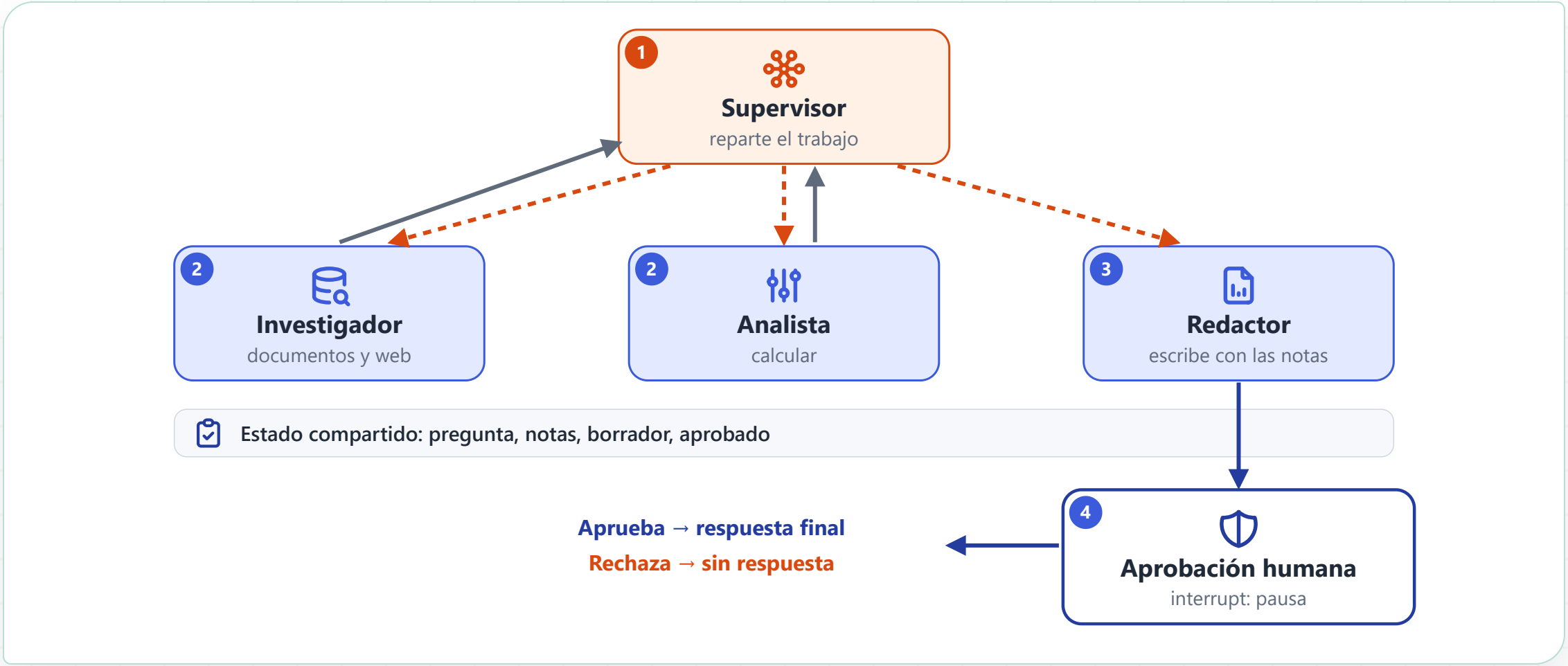
qué paso sigue, a veces según una decisión



### Estado

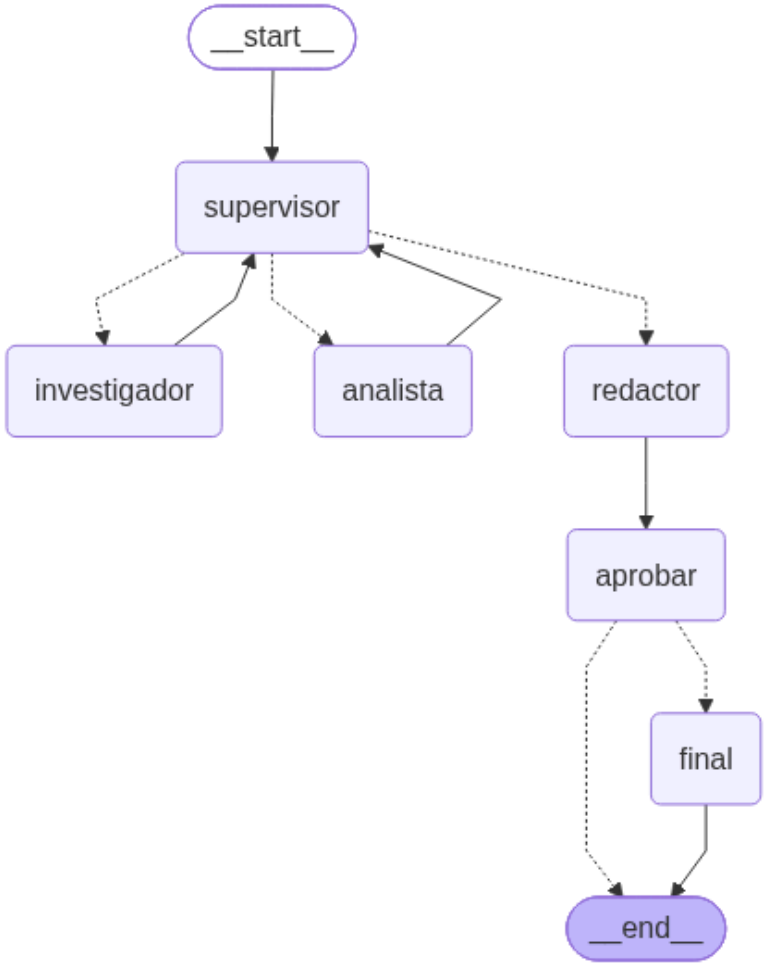
un diccionario que pasa de caja en caja, como una carpeta

# v0.5: un supervisor, tres especialistas, una firma




El estado viaja entre ellos. Una persona decide al final.

# Así dibuja LangGraph tu v0.5: lo punteado se decide

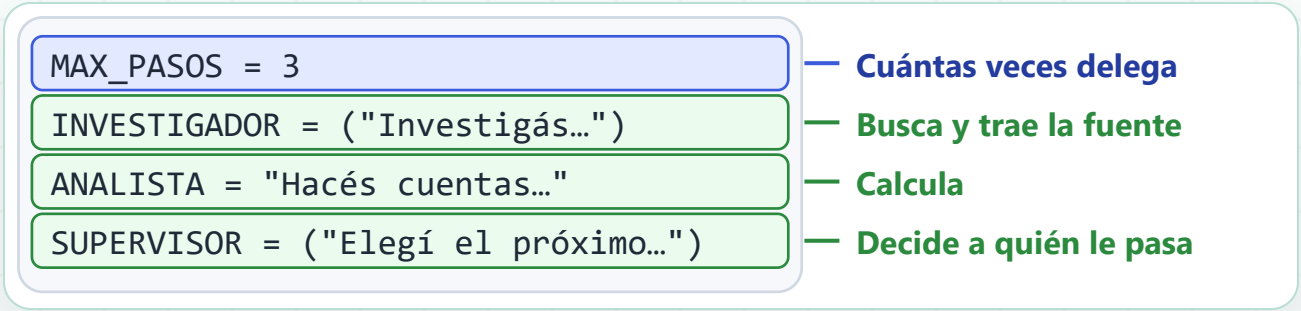


# Taller: corré y dibujá tu grafo (60 min)

1. **Corré**

   ```
   uv run python v05_equipo.py
   ```

2. **Dibujo** en mermaid.live.
3. **Notas** de cada uno.




Práctica M7, arranque y pasos 1 a 3.

**Primer control · 10–15 min:** grafo, borrador y dos respuestas.

# La persona que aprueba es el último control

**Borrador del modelo chico**
**«la tabla tiene 15 filas; ×3 = 45»**

×

**Rechazo**
**sin respuesta final**

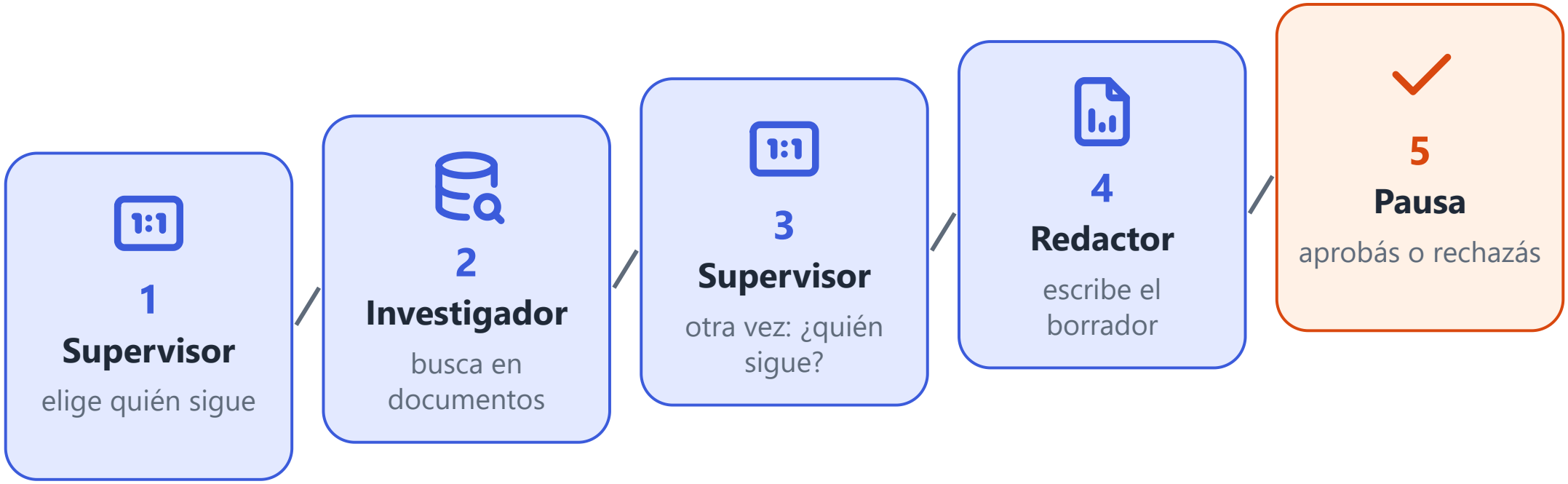
- Son 5 filas de comparación (más una de enlaces), no 15.
- ¿Quién lo detecta?

# Con Gemini acertó, pero salteó al analista

```
[supervisor]   {'siguiente': 'investigador', 'pasos': 1}
[investigador] {'notas': ['Según la tabla comparativa entre Checkpointer y Store…
[supervisor]   {'siguiente': 'redactor', 'pasos': 2}
[redactor]     {'borrador': 'Según la documentación, la tabla … tiene 5 filas…
[PAUSA]        espera a una persona · borrador:
               … tiene 5 filas de contenido. Multiplicando ese número por 3,
               el resultado es 15.
[aprobado]     respuesta final: '… el resultado es 15.'
[rechazo]      respuesta final: (ninguna: el rechazo la bloqueó)
```

La cuenta la hizo el redactor, sin calculadora: salió bien, pero nadie la comprobó. ▶ `uv run python -m ejemplos.clase7 equipo`

# La ruta que tomó: el analista no se llamó




El supervisor decide la ruta en cada consulta: esta vez, sin analista.

¿Cómo harías que pida al analista cuando hay una cuenta?

Paso 4 de 5: Aprobación humana

# El grafo se pausa y retoma

m7_v05_equipo.ipynb

```python
def aprobar(e):
    decision = interrupt({"borrador": e["borrador"]})  # PAUSA: el estado queda guardado
    return {"aprobado": bool(decision.get("aprobado", False)) if isinstance(decision, dict) else
decision is True}

def correr(pregunta, hilo, aprobado, contador=None):
    config = {"configurable": {"thread_id": hilo}, "callbacks": [contador] if contador else []}
    pausa = grafo.invoke({"pregunta": pregunta}, config)
    final = grafo.invoke(Command(resume={"aprobado": aprobado}), config)  # mismo hilo = mismo estado
    return pausa, final
```

`interrupt` pausa, `Command(resume=…)` retoma el mismo hilo.

# Taller: aprobar, rechazar y comparar (40 min)

1. **Aprobar y rechazar:** `True` / `False` en `correr`
2. **Rechazo** = sin respuesta final.
3. **v0.4 vs v0.5**, misma suite.

```
correr(pregunta, "demo-aprueba", True)
correr(pregunta, "demo-rechaza", False)
```

— **True: la persona aprueba**

— **False: rechaza, sin respuesta final**

Práctica M7, pasos 4 a 5.

**Trampa:** ¿el rechazo responde? Mirá `aprobar`.

Paso 5 de 5: Comparar

# ¿Qué cambiarías en el supervisor?

**¿El modelo?**

uno más grande

**¿Las instrucciones?**

lo que dice SUPERVISOR

**¿El tope de pasos?**

MAX_PASOS

**Elegí una y decí por qué.**

El supervisor del modelo chico mandó preguntas de web al analista.

# CrewAI piensa en roles, LangGraph, en estado

| | CrewAI | LangGraph |
|---|---|---|
| Se arma con | Roles | Estado y nodos |
| Quién decide el orden | El proceso: secuencial o jerárquico | Vos, con aristas |
| Pausa humana | `human_input=True` en una tarea | `interrupt` en un nodo |

# Repaso: hoy te llevás tres ideas

### Equipo

un supervisor reparte

### Grafo

cajas, flechas y estado

### Aprobación

una persona aprueba al final

**Más agentes no siempre es mejor: medí el costo antes de sumarlos.**

# ¿Qué pasa del investigador al redactor?

Clínica: mostrá tu tabla «agente solo vs equipo» y la prueba de rechazo.

¿Qué información pasa del investigador al redactor, y por qué esa y no toda?

Anotá tu **confianza (1 a 4)**.